Add Value to Your Geospatial Plots

Hendrix Peralta · Follow

6 min read · Aug 27, 2024

Listen Share

Plot coordinate points and add multiple legends

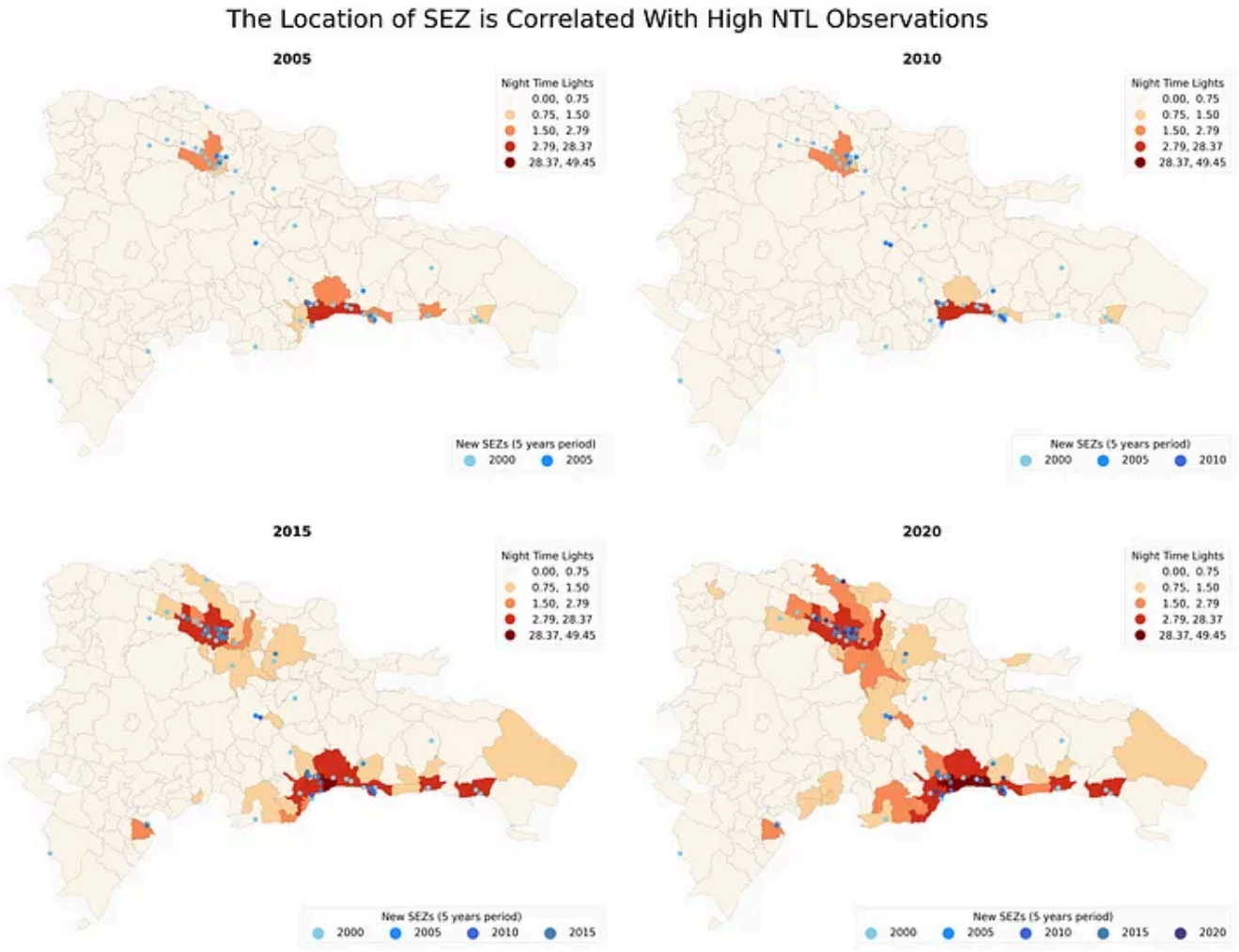


Figure 1. Distribution of NTL and SEZ locations. Source: Author

Imagine that we have a dataset with a country's Night Time Lights (NTL) and its Special Economic Zones (SEZ) location. Subsequently, we found a correlation between the SEZ and NTL and wanted to show it to our colleagues.

However, when creating this new Figure using geospatial data, we might think that the basic plotting function will produce something that looks like Figure 1. But then we realized that our graph looked more like Figure 2 and became disappointed.

```
fig, ax = plt.subplots()

# Plot GeoDataFrame
geo_municipalities.plot(
    column="ntl_2020",          # Column to plot
    scheme="natural_breaks",    # Data Classification scheme
    cmap="OrRd",                # Color map
    edgecolor="k",
    legend=True,                # Show legend
    ax=ax
)
```

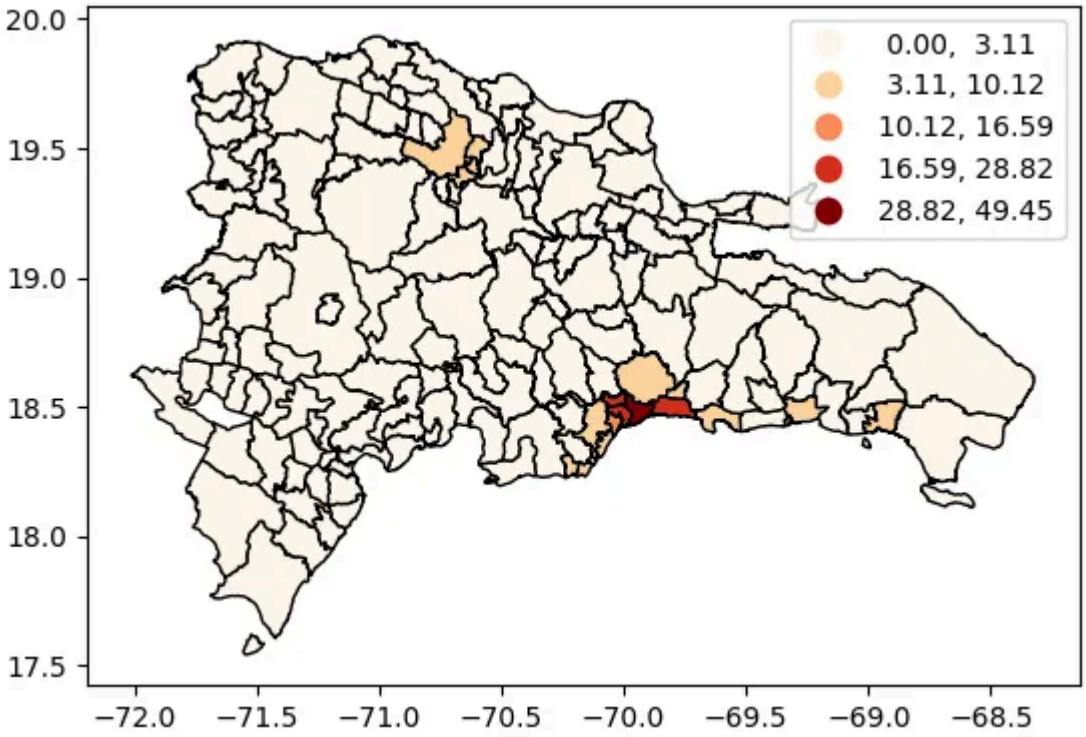


Figure 2. Figure with the basic parameters. Source: Author

This article will explain the steps needed to improve your plots by transforming plots that look like Figure 2 into ones similar to Figure 1.

1. Plot parameters and cluster classification

Our first step will be to modify some of its parameters to make our Figure more appealing. In this case, we will eliminate the axis, reduce the plot line thickness,

and add a title.

Additionally, we are going to change the classification scheme. I prefer the classification algorithm produced by GEODA over the built-in classification options. The reason is that the patterns shown in GEODA offer more information. We can install the pygoeda library to change the classification scheme and create our new natural brakes.

```
# Create new natural brakes using pygoeda
breaks = pygeoda.natural_breaks(6, geo_municipalities["ntl_2020"])

fig, ax = plt.subplots()

geo_municipalities.plot(
        column = "ntl_2020",
        scheme="UserDefined",  # Change the classification scheme
        classification_kwds={"bins":breaks}, # Brakes creaded with pygoeda
        cmap="OrRd",
        edgecolor="k",
        linewidth=0.2, # Thinner line width
        legend=True,
        ax=ax)

ax.set_axis_off();
ax.set_title("NTL Distribution",
            pad=15,
            fontsize=10,
            fontdict={"weight":"bold"});
```

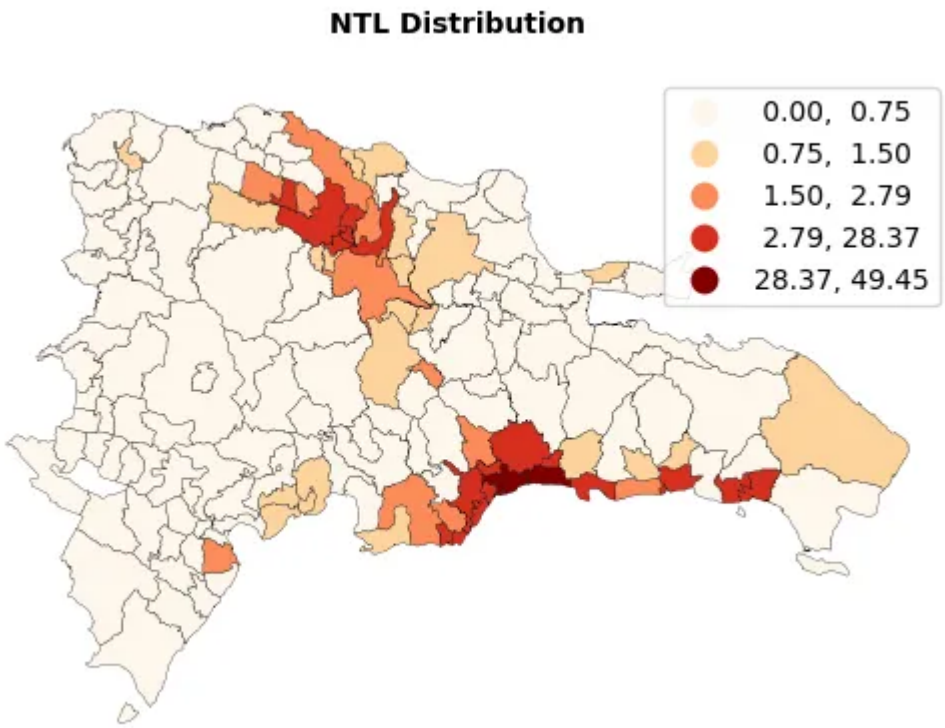


Figure 3. Changed basic parameters and classification scheme. Source: Author

2. Add coordinate points to a map

The most important thing when combining two geospatial data into one graph is to confirm that their coordinates are the same. You can use the method .crs on both geopandas objects to do this.

```
geo_municpalities.crs

output:

<Geographic 2D CRS: GEOGCS["GCS_unknown",DATUM["WGS_1984",SPHEROID["WG ...>
Name: GCS_unknown
Axis Info [ellipsoidal]:
- lon[east]: Longitude (Degree)
- lat[north]: Latitude (Degree)
Area of Use:
- undefined
Datum: World Geodetic System 1984
- Ellipsoid: WGS 84
- Prime Meridian: Greenwich
```

After confirming that the coordinates are equal, you can plot the SEZ location data and assign it to the same axis as the NTL distribution graph.

```
breaks = pygeoda.natural_breaks(6, geo_municipalities["ntl_2020"])

fig, ax = plt.subplots()

geo_municipalities.plot(
        column = "ntl_2020",
        scheme="UserDefined",
        classification_kwds={"bins":breaks},
        cmap="OrRd",
        edgecolor="k",
        linewidth=0.2,
        legend=True,
        ax=ax)

# Coordinate points plot
sez_location_2020.plot(
            ax=ax,   # Assigning the second plot to the same axis
            color="yellow",
            marker="o",
            markersize=10)

ax.set_axis_off();
ax.set_title("NTL Distribution and SEZ location",
            pad=15,
            fontsize=10,
            fontdict={"weight":"bold"});
```

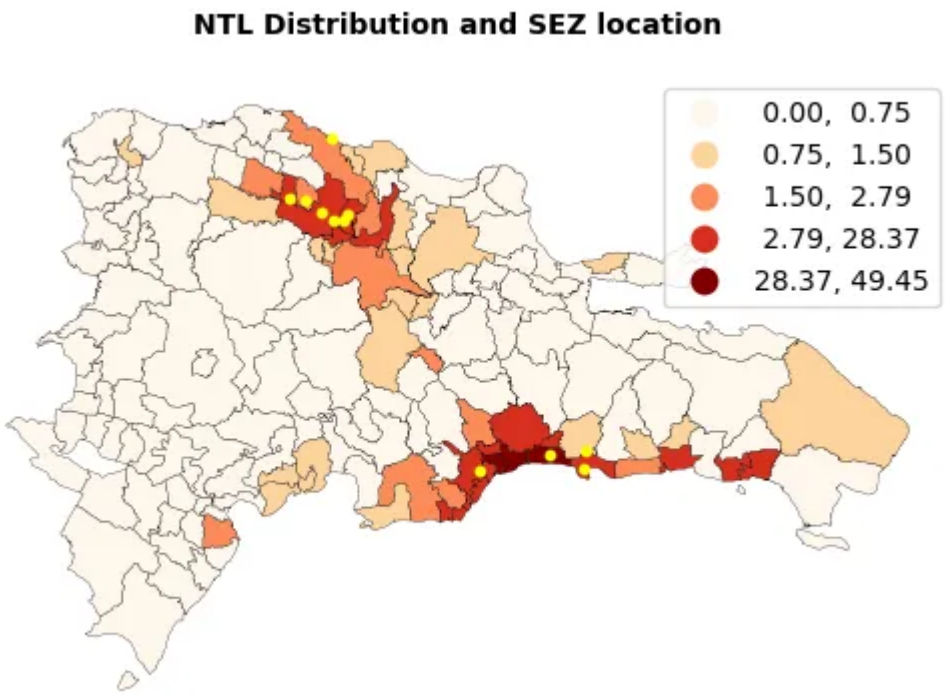


Figure 4. NTL distribution and SEZ opened between 2015 and 2020. Source: Author

3. Subplots and Overall Title

Now, we have a graph that shows us the NTL distribution for 2020 and the location of the SEZ opened in the 5 years from 2015 to 2020. However, seeing the changes in NTL over time would be more insightful. This way, we can see if there are any patterns in the data over time. To achieve this, we will create four subplots for 2005, 2010, 2015, and 2020.

To avoid repeating the same code for each plot, we will create a function that receives only the unique parameters of each plot. This way, we can make modifications in one place when we want to change other aspects of the plots that all of them share, such as title, legend, and maker sizes.

Additionally, to add an overall title on top of all your subplots, you can use the method .suptitle() and modify its parameters.

```
breaks = pygeoda.natural_breaks(6, geo_municipalities["ntl_2020"])

# Recieves the unique parameters of each plot
def distributionPlusCoordinatePoints(geo_map, column, breaks,
                                     coordinates, title=None,
                                     period=None, ax=None):

    # geo_map = geoDataFrame for the cluster classification
    geo_map.plot(
        column = column,
        scheme="UserDefined",
        classification_kwds={"bins":breaks},
        cmap="OrRd",
        edgecolor="k",
        linewidth=0.2,
        legend=True,

        # Scales the legend
        legend_kwds={"fontsize":30,
                     "markerscale":3,
                     "title":"Night Time Lights",
                     "title_fontsize":30}, #Scales the items inside the legend
        ax=ax,
    )

    # Coordinates = Coordinate point to plot on top of the map
    coordinates.plot(
        ax=ax,
        color="black",
        marker="o",
        markersize=190, # Scales the marker size
```

```
            label=year,
        )

        ax.set_axis_off();
        ax.set_title(title,
                 pad=15,
                 fontsize=40,
                 fontdict={"weight":"bold"});
    # ================= Function ends =================


    # Creates a 2x2 subplot array
    fig, axes = plt.subplots(2, 2, figsize=(50, 40))
    ax1, ax2, ax3, ax4 = axes.flat


    distributionPlusCoordinatePoints(geo_municipalities,
                                     "ntl_2005",
                                     breaks,
                                     coordinates=sez_location_2005,
                                     title="2005",
                                     ax=ax1
                                     )

    distributionPlusCoordinatePoints(geo_municipalities,
                                     "ntl_2010",
                                     breaks,
                                     coordinates=sez_location_2010,
                                     title="2010",
                                     ax=ax2
                                     )

    # Repeate for each of the subplots
    distributionPlusCoordinatePoints(...)

    distributionPlusCoordinatePoints(...)

    # creates an overall title
    fig.suptitle("The Location of SEZ is Correlated With High NTL Observations",
                    fontsize=70,
                    fontdict={"weight":"bold"});

    plt.tight_layout()
    plt.show()
```

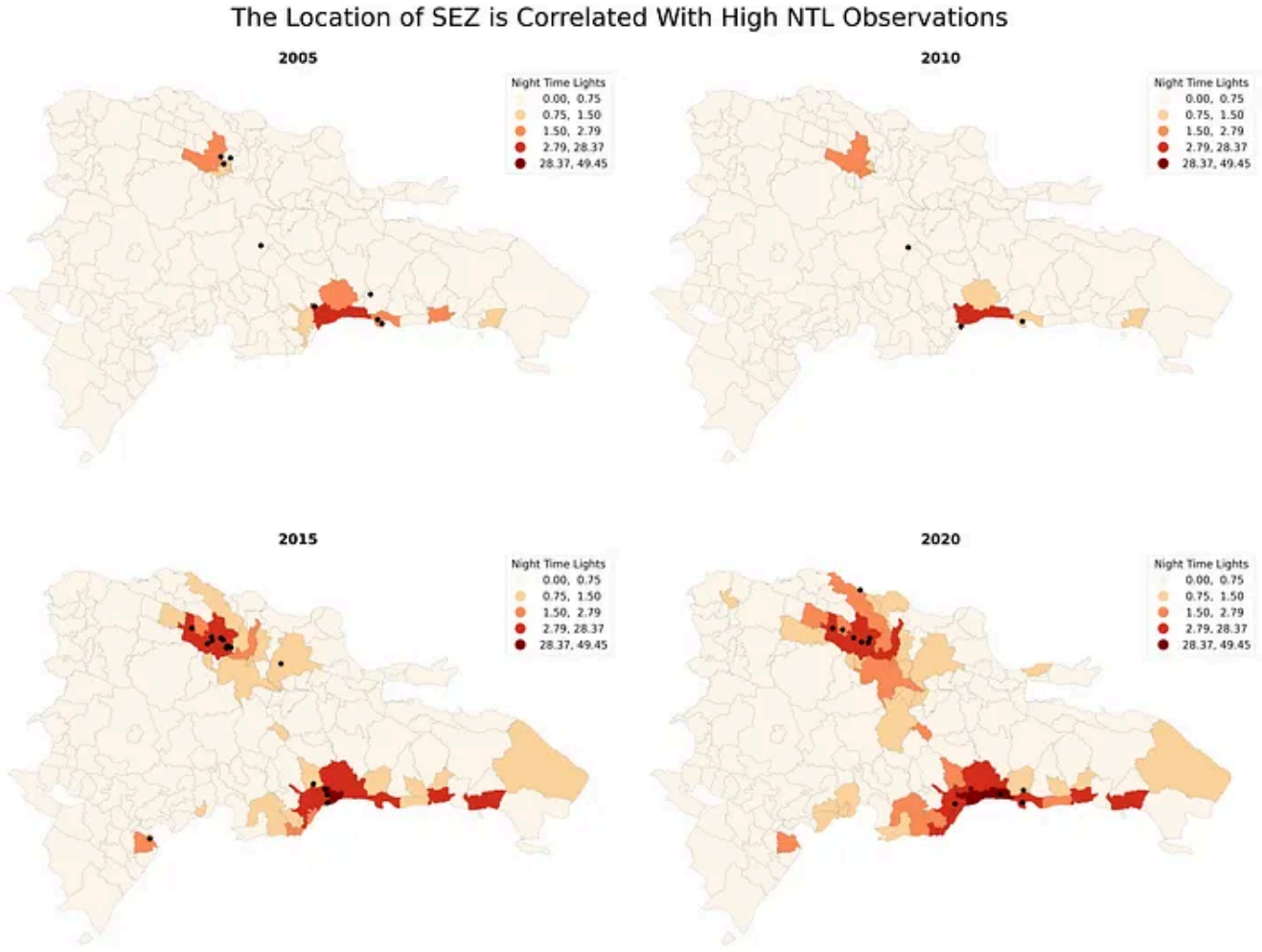


Figure 5. Distribution of NTL and SEZ opened from 2005 to 2020. Source: Author

4. Add a second legend to your Plots

For the next step, we will change how we present the SEZ location information to finish our plot. Instead of only showing the SEZ that opened in a 5-year period, we will also show the previous SEZ and add a legend that clarifies in which period they were opened.

To do this, we will pass a list of coordinates to the distributionPlusCoordinatePoints() function and iterate on it to plot the sez location in each period.

Finally, we will add a second legend to the subplots to clarify the period in which the SEZ was opened. When we add the second legend, the first one will disappear, and we need to bring it back using the .add_artist() method.

```
breaks = pygeoda.natural_breaks(6, geo_municipalities["ntl_2020"])

def distributionPlusCoordinatePoints(geo_map, column, breaks,
                                     coordinates, title=None,
```

```
                                    period=None, ax=None):

    # Color to be used for the coordinated plot
    color_list = ["#87CEEB", "#1E90FF", "#4169E1", "#4682B4", "#483D8B"]
    # Labels for each of the coordinate points
    years = [2000, 2005, 2010, 2015, 2020]

    geo_map.plot(
        column = column,
        scheme="UserDefined",
        classification_kwds={"bins":breaks},
        cmap="OrRd",
        edgecolor="k",
        linewidth=0.2,
        legend=True,
        legend_kwds={"fontsize":30,
                     "markerscale":3,
                     "title":"Night Time Lights",
                     "title_fontsize":30}, #Scales the items inside the legend
        ax=ax,
    )

    # Save the natural brakes legend in a variable
    leg1 = ax.get_legend()

    # Iterate on the sez location, color and label lists
    for coordinate, color, year in zip(coordinates,color_list, years):
        coordinate.plot(
            ax=ax,
            color=color, # assigns a different color to each period
            marker="o",
            markersize=170,
            label=year, # assigns a different label to each period

            # Change parameters the second legend
            legend_kwds={"fontsize":30,
                    "markerscale":2.5,
                    "loc":"lower right"}
        )

    # Creates the second legend (the first legend is deleted)
    ax.legend(title='New SEZs (5 years period)',
                fontsize=30,
                title_fontsize='30',
                markerscale=2.5,
                loc='lower right', # location of the legend
                ncol=5)

    ax.add_artist(leg1) # adds the first legend to the plot

    ax.set_axis_off();
    ax.set_title(title,
             pad=15,
```

```
fontsize=40
ax1, ax2, ax3, ax4 = axes.flat

distributionPlusCoordinatePoints(geo_municipalities,
                                 "ntl_2005",
                                 breaks,
                                 # Passes a list of coodinates points
                                 coordinates=[sez_location_2000,
                                              sez_location_2005],
                                 title="2005",
                                 ax=ax1
                                 )

distributionPlusCoordinatePoints(geo_municipalities,
                                 "ntl_2010",
                                 breaks,
                                 coordinates=[sez_location_2000,
                                              sez_location_2005,
                                              sez_location_2010],
                                 title="2010",
                                 ax=ax2
                                 )

# Repeat for each subplot
distributionPlusCoordinatePoints(...)

distributionPlusCoordinatePoints(...)

fig.suptitle("The Location of SEZ is Correlated With High NTL Observations",
                fontsize=70,
                fontdict={"weight":"bold"});

plt.subplots_adjust(wspace=0.1)
plt.tight_layout()
plt.show()
```

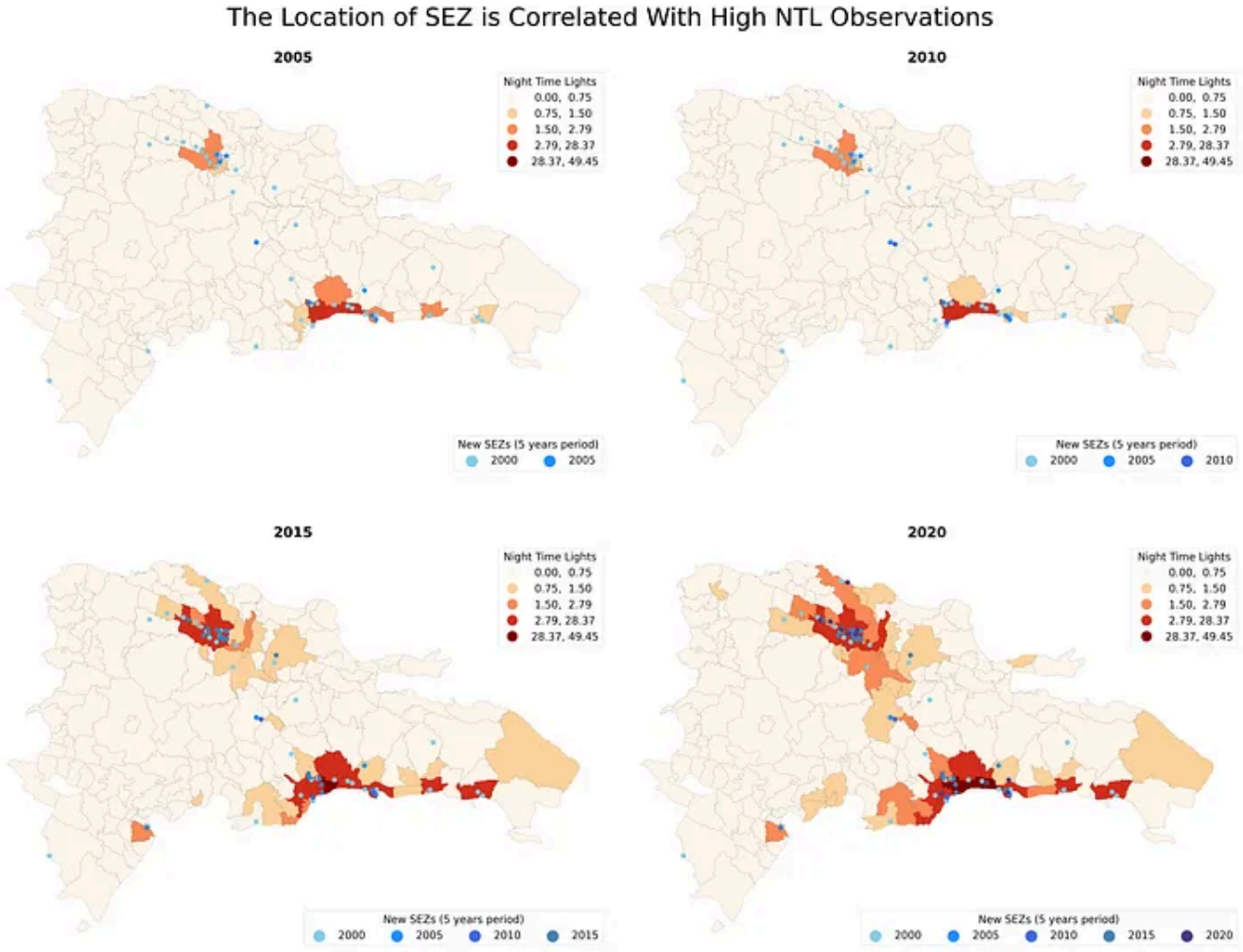


Figure 6. Final plot. Source: Author

Geopandas Documentation: Choropleth classification schemes from PySAL for use with GeoPandas — GeoPandas 1.0.1+0.g747d66e.dirty documentation

Geographic Data Science with Python (online book): Choropleth Mapping — Geographic Data Science with Python

Data Science | Geopandas | Python | Data Visualization | Matplotlib

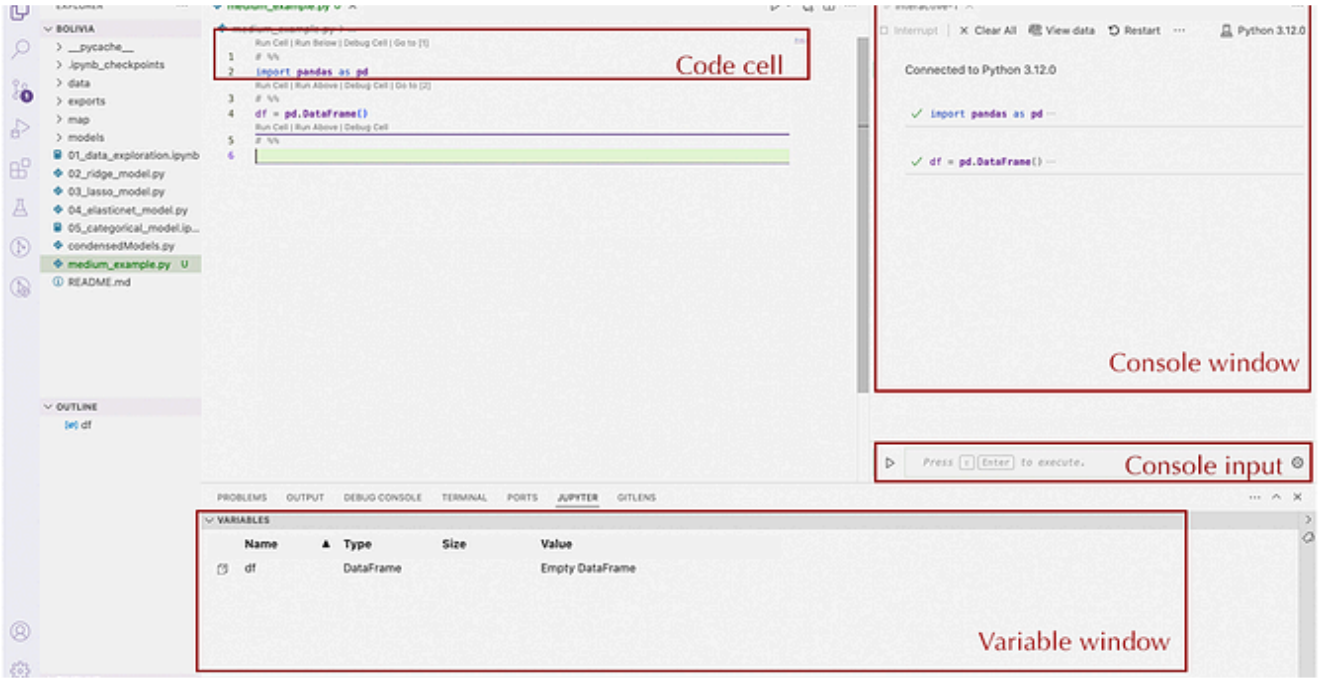
Follow

Written by Hendrix Peralta

0 Followers

Master's in Economic Development with a passion for Data science. My research focuses on regional development, satellite data, and machine learning.

More from Hendrix Peralta



Hendrix Peralta

Improve Your Data Analysis Workflow

Replace notebooks with Python interactive files -> profit.

Aug 24

See all from Hendrix Peralta

Recommended from Medium

Sangeeth Joseph - The AI dev in Python in Plain English

5 Modern Python Tools Every Developer Should Master in 2024

Introduction

Sep 12 134 2

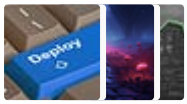
Datylon

Which Data Visualization Tool Is Best For Dashboards?

by Julia Vorontsova—Chief Marketing Officer @ Datylon

Aug 9 75 1

Lists



Predictive Modeling w/ Python

20 stories · 1541 saves

Coding & Development

11 stories · 814 saves

Practical Guides to Machine Learning

10 stories · 1870 saves

ChatGPT prompts

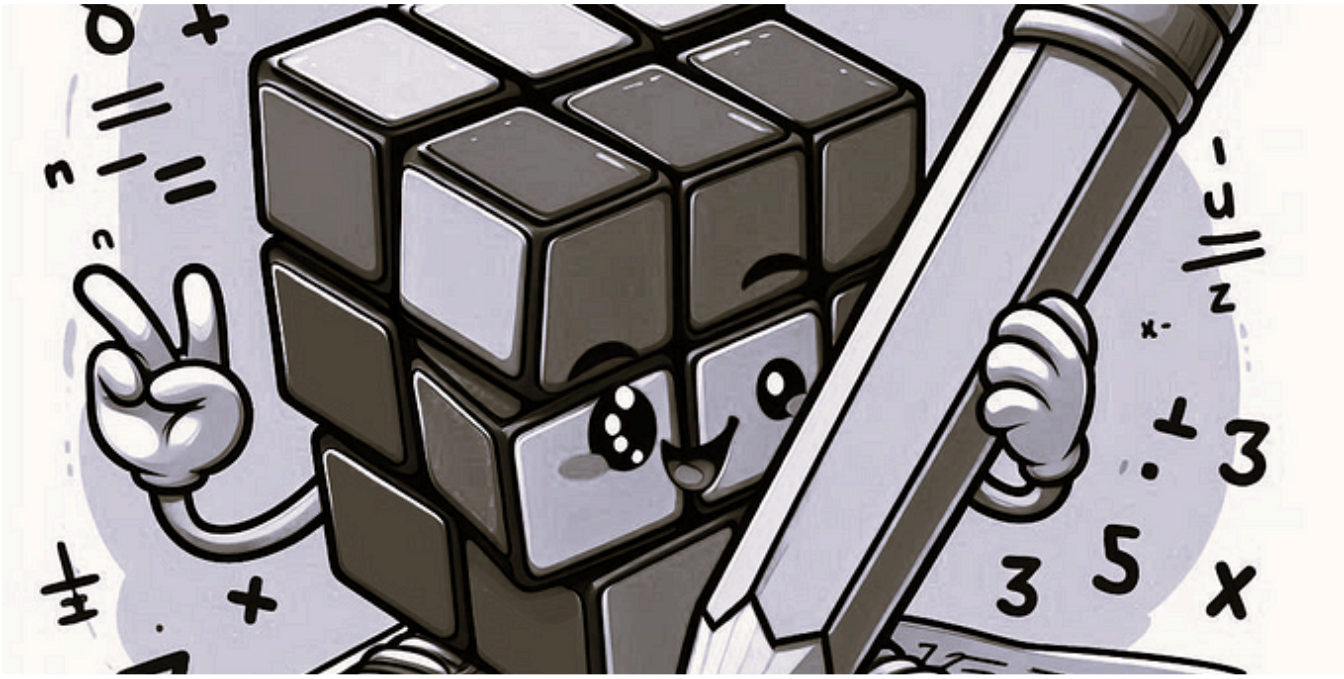
48 stories · 1999 saves

Pralabh Saxena in Level Up Coding

12 Useful Python Libraries That You May Not Know About

Useful Python libraries that you would love to try

4d ago 127

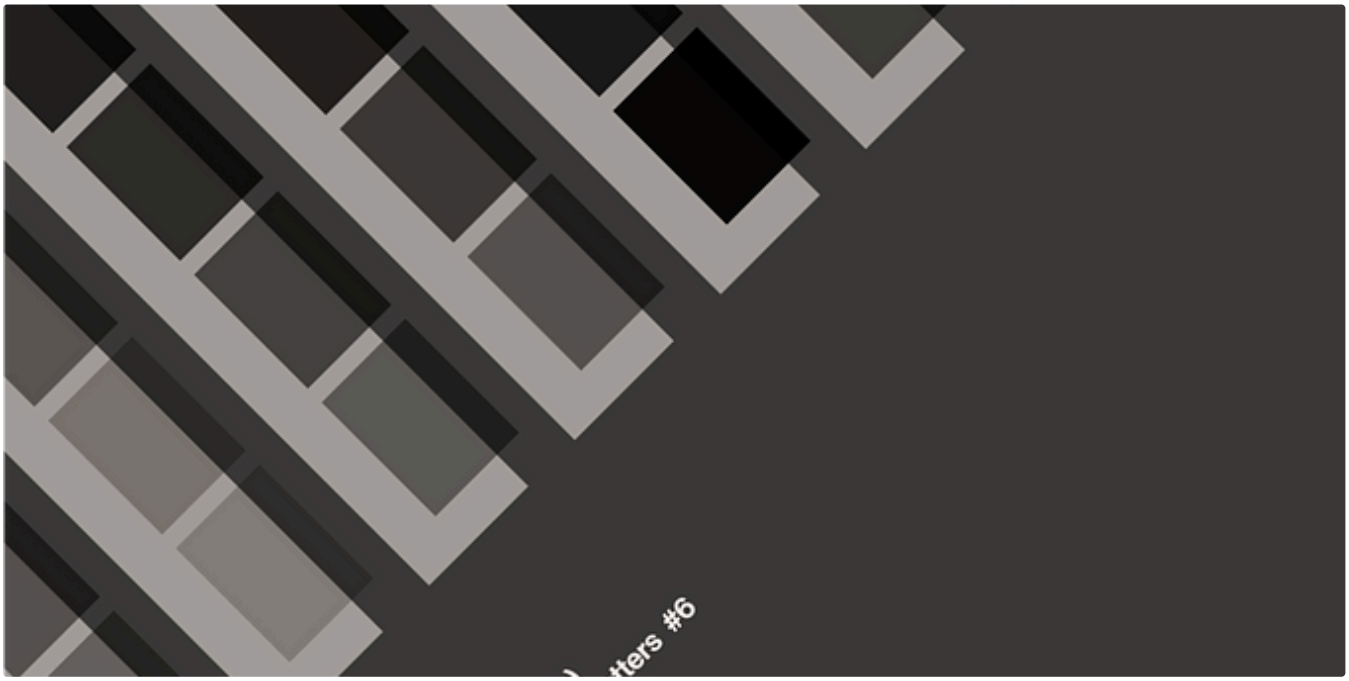


Lee Vaughan in Towards Data Science

Introducing NumPy, Part 4: Doing Math with Arrays

Plus reading and writing array data!

3d ago 293 2

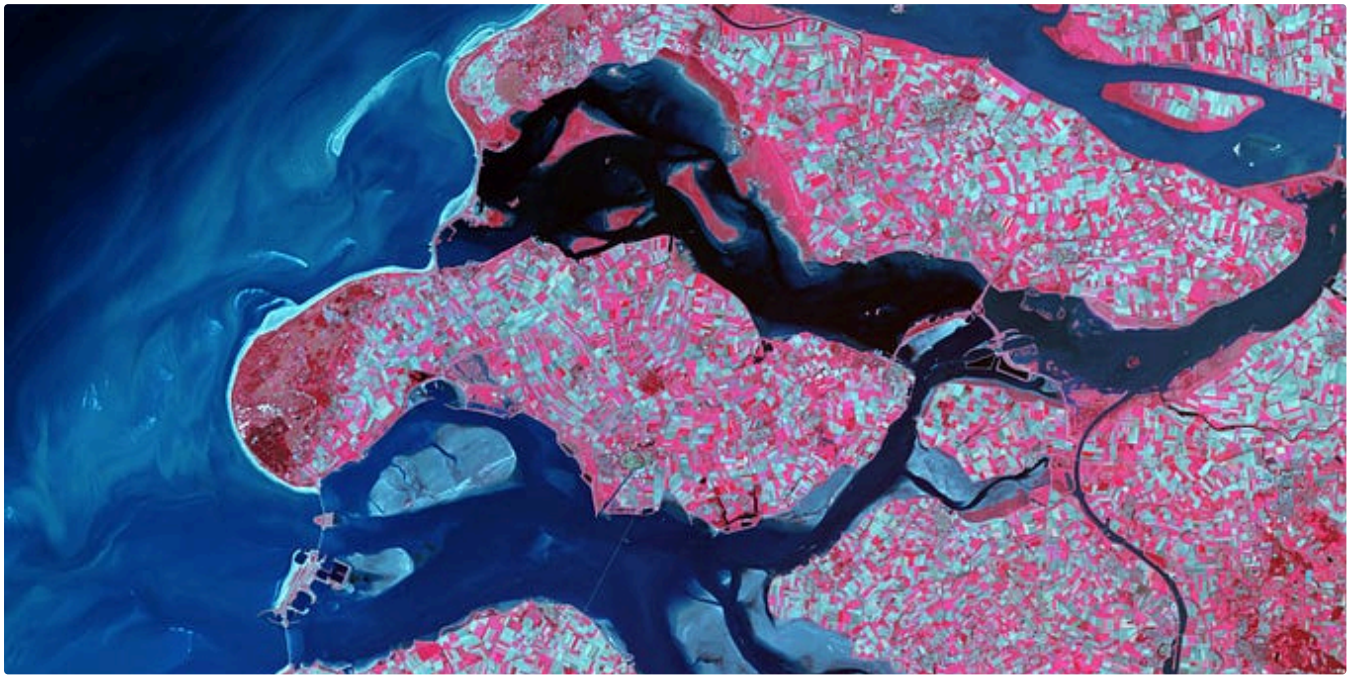


T. from Data Rocks

Design Matters #6 — The Ultimate Dashboard Colour Palette

This is a repost of the Design Matters series I run as part of my newsletter, originally sent to my subscribers on 18 August 2023.

Aug 28 270 2



KokaTic in Stackademic

Top Sources for Satellite and GIS Data: A Comprehensive Guide

Exploring Key Platforms for Accessing and Utilizing Satellite Imagery and Geographic Information Systems

Aug 28 35